

Personal Finance Tracker

DATA 333 Project Report

A Python application to track, analyze and visualize personal finances
with pandas, Tkinter and Streamlit

DATA 333 – Data Management & Analysis

Bellevue College — Prior Learning Assessment (PLA)

Student Name: _____

October 1, 2026

1 Project overview and objectives

The Personal Finance Tracker records income and expenses, organizes them into categories, and turns them into summaries, trends, budget alerts and savings-goal progress. One tested core is used by three interfaces: an interactive **console menu** (`python main.py cli`), a **Tkinter desktop application** (`python main.py gui`) and an interactive **Streamlit dashboard** (`streamlit run finance_tracker/dashboard.py`). A non-interactive **demo mode** (`python main.py demo`) runs every feature on a reproducible sample of 311 fictional transactions (2026-04 to 2026-09). **Requires Python 3.11+.**

The objectives, taken from the course brief, were to: track income and expenses; categorize transactions; display spending summaries; set and track savings goals; save and load data from files; analyze spending trends; visualize the results — while demonstrating every core concept of the course (input/output, decisions and loops, functions, files and exceptions, lists/dictionaries/sets, classes, pandas) and implementing all four enhancements: Tkinter GUI, JSON *and* CSV storage, budget alerts and a Streamlit dashboard.

2 Key features

- **Transactions** — add, edit, delete, text search and filters (kind, category, date range, amount); every value is validated before it is stored.
- **Summaries and trends** — monthly income, expenses, net and savings rate, category shares, a 3-month rolling average, month-over-month change, top expenses.
- **Budgets and alerts** — a monthly limit per category; *WARNING* at 80 %, *EXCEEDED* at 100 %, raised as soon as a new expense crosses a limit.
- **Savings goals** — target, deadline, contributions and milestones at 25/50/75/100 %.
- **Files** — full state in JSON, transactions in CSV (import/export), atomic writes, and recovery from a corrupted file.

All figures below are real captures of the running application on the sample data, produced by `tools/capture_screenshots.py`; GUI and dashboard captures are cropped to the region being discussed (full-size images are in `docs/figures/`).

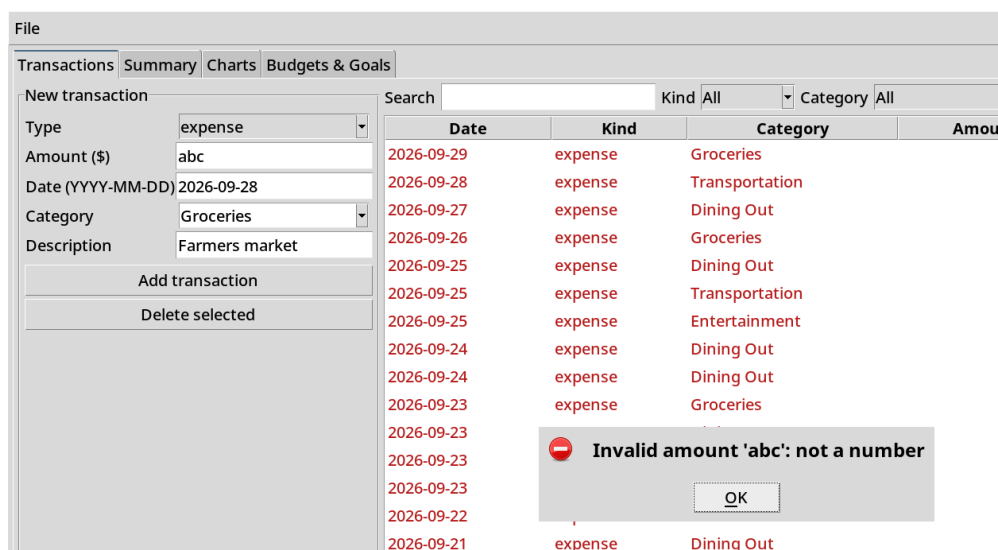


Figure 1: Tkinter desktop application (`python main.py gui`): the entry form, the searchable and sortable transaction table, and the message shown when an invalid amount (`abc`) is submitted — nothing is stored.

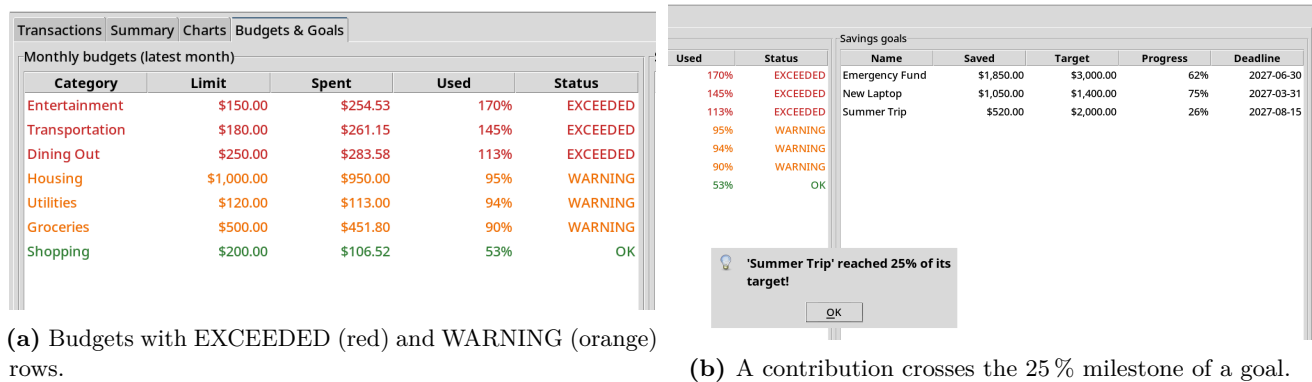


Figure 2: Budget alerts and savings goals in the Tkinter application.

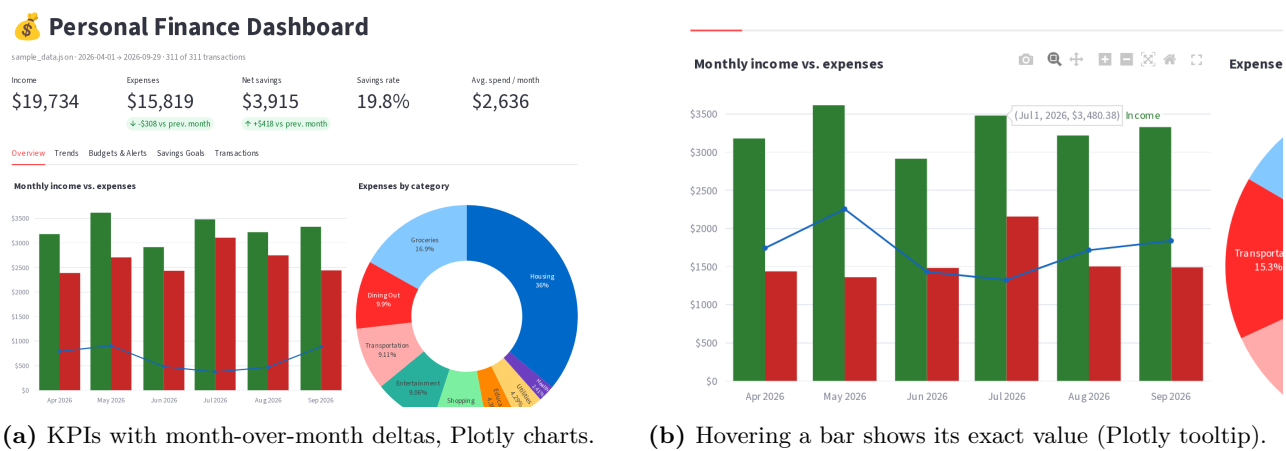


Figure 3: Streamlit dashboard, captured with headless Chromium (Playwright).

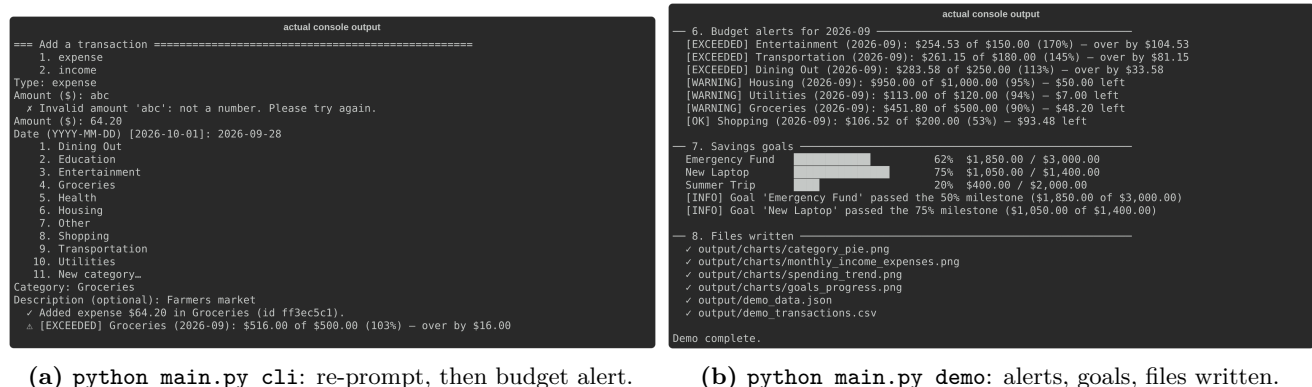


Figure 4: Console modes — actual console output, recorded with rich.

Figures 1 and 2 show the desktop application, Figure 3 the web dashboard and Figure 4 the two console modes; the same alerts appear in all three interfaces because they come from the same functions.

3 How course concepts were applied

Every concept is tagged in the source with a `# Concept:` comment at the exact line where it is applied; a test (`tests/test_code_standards.py`) fails if one is missing.

Concept	File / function	Excerpt
Input / output	<code>cli.py</code> – <code>ConsoleApp.ask</code> , <code>show_summary</code>	<code>self._input(prompt).strip()</code> ; aligned f-string tables
Decisions & loops	<code>alerts.budget_level</code> , <code>ConsoleApp.run</code> , <code>prompt_amount</code>	<code>if ratio >= EXCEEDED_THRESHOLD: ... elif ...;</code> <code>while True: re-prompt</code>
Functions	<code>models.parse_amount</code> , <code>analytics.*</code>	small, typed, documented functions reused by every interface
Files & exceptions	<code>storage.load_json</code> , <code>_atomic_write</code>	<code>try/except/else; try/except/finally</code> ; corrupted file backed up
Lists / dicts / sets	<code>FinanceTracker.__init__</code>	<code>transactions: list, budgets: dict, categories: set</code>
Classes (OOP)	<code>Transaction</code> , <code>SavingsGoal</code> , <code>Budget</code> , <code>FinanceTracker</code>	dataclasses validated in <code>__post_init__</code> ; properties; inheritance
pandas	<code>analytics.monthly_summary</code> , <code>spending_trend</code>	<code>pivot_table</code> , <code>groupby().agg()</code> , <code>rolling(3).mean()</code> , <code>pct_change()</code>

Three excerpts, extracted automatically from the real code (docstrings omitted):

Listing 1: `cli.py` — loop, decision, exception

```
def prompt_amount(self, prompt: str) -> float:
    # Concept: loop + decision -- re-prompt
    ↪ until the user enters a valid amount
    while True:
        raw = self.ask(prompt)
        try:
            return parse_amount(raw)
        except ValidationError as exc:
            self.out(f" × {exc}. Please try
            ↪ again.")
```

Listing 2: `alerts.py` — decision structure

```
def budget_level(ratio: float) -> str:
    # Concept: decision structure -- thresholds
    ↪ are tested from most to least severe
    if ratio >= EXCEEDED_THRESHOLD:
        return LEVEL_EXCEEDED
    elif ratio >= WARNING_THRESHOLD:
        return LEVEL_WARNING
    else:
        return LEVEL_OK
```

Listing 3: `analytics.py` — pandas groupby

```
def category_breakdown(df: pd.DataFrame, kind:
    ↪ str = EXPENSE) -> pd.DataFrame:
    subset = df[df["kind"] == kind]
    if subset.empty:
        return
    ↪ pd.DataFrame(columns=["category",
    ↪ "total", "share", "count"])
    # Concept: pandas -- groupby + named
    ↪ aggregation computes several stats at
    ↪ once
    grouped = (
        subset.groupby("category")
        .agg(total=("amount", "sum"),
        ↪ count=("amount", "size"))
        .sort_values("total", ascending=False)
        .reset_index()
    )
    grouped["share"] = grouped["total"] /
    ↪ grouped["total"].sum() * 100
    return grouped[["category", "total",
    ↪ "share", "count"]].round(2)
```

4 Architecture

The interfaces contain no business rules: they call `FinanceTracker` and the `analytics`, `alerts` and `visualize` modules, and only format the results. Validation lives in the model classes and every file format lives in `storage.py`. Figure 5 shows the modules and how data flows between them.

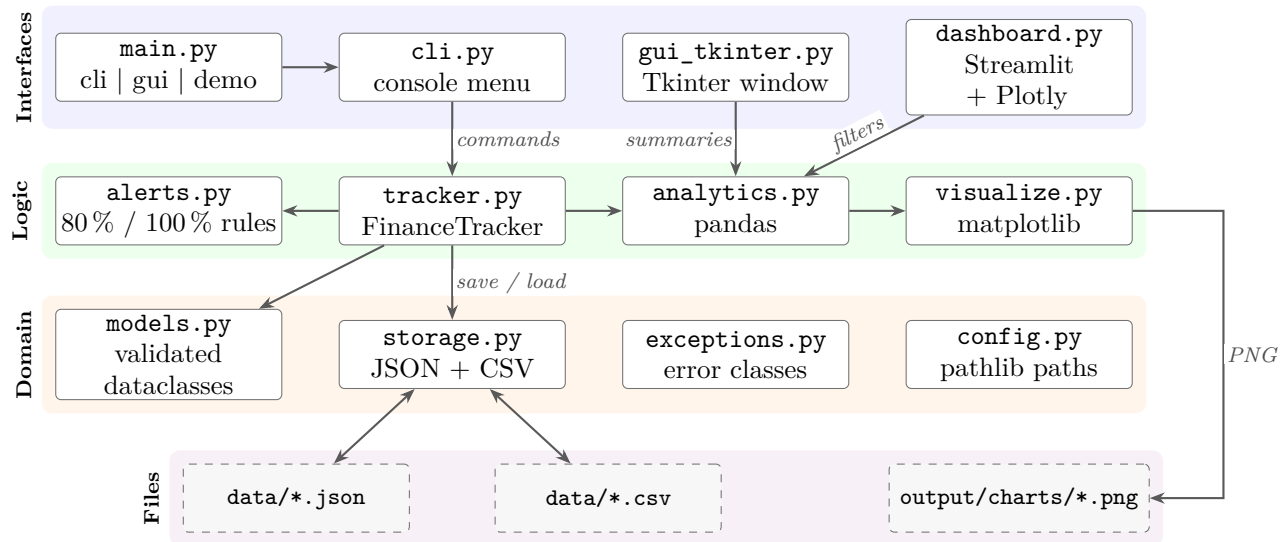


Figure 5: Modules and data flow. `main.py` starts one interface; interfaces send commands to `FinanceTracker`, which holds validated model objects and delegates every file format to `storage.py`. The tracker passes budgets and spending to `alerts` and transactions to `analytics`, whose pandas DataFrames feed tables, KPIs and the `visualize` charts (PNG files, or figures embedded in the GUI).

5 Libraries used

Library	Version	Role in the project
Python	3.11.16	language and standard library (3.11+ required)
pandas	3.0.6	DataFrames, <code>pivot_table</code> , <code>groupby</code> , rolling averages
matplotlib	3.11.2	PNG charts and figures embedded in the Tkinter window
plotly	7.1.0	interactive charts (tooltips, zoom) in the dashboard
streamlit	1.64.0	web dashboard: filters, KPIs, tabs, CSV upload/download
tkinter / ttk	Tk 8.6	desktop GUI: notebook tabs, form, <code>Treeview</code>
json, csv	stdlib	JSON state file and CSV import/export
pathlib	stdlib	portable paths, no hard-coded absolute path
dataclasses	stdlib	<code>Transaction</code> , <code>SavingsGoal</code> , <code>Budget</code> with validation
pytest, pytest-cov	9.1.1, 7.1.0	automated tests and coverage measurement
ruff	0.16.9	linter and formatter (code style)
rich, Playwright	15.0.0, 1.63.0	reproducible captures of the console and dashboard

6 Challenges and solutions

Real problems met during development, taken from DEV_LOG.md:

Challenge	Solution
Our <code>StorageError</code> (a subclass of <code>OSError</code>) was caught by a generic <code>except OSError</code> and re-wrapped, hiding the real message.	<code>except StorageError: raise</code> placed <i>before</i> the generic clause: handlers are matched in order and by inheritance.
Labels such as “\$1,850 / \$3,000” lost their dollar signs: matplotlib and <code>st.markdown</code> render text between two \$ signs as math.	Escape every \$ (<code>\\$</code>); regression tests on chart labels and dashboard messages.
CLI tests read the real keyboard: <code>input_func=input</code> was a default argument, evaluated once when the function is defined.	Default <code>None</code> , resolved at call time; tests inject scripted input.
On a HiDPI screen the Tkinter table rows overlapped and chart text was tiny.	Scale factor computed from the real font height; charts drawn at the screen’s dpi.
An independent review found that “0.004” passed the <code>> 0</code> check <i>before</i> rounding, then crashed with <code>ZeroDivisionError</code> .	Round first, then validate (≥ 0.01); regression tests for budgets, goals and transactions.
Valid but wrongly shaped JSON (<code>{"transactions": null}</code>) crashed all interfaces.	<code>_check_structure()</code> treats it like a syntax error: the file is backed up and the application starts.

7 Enhancements implemented

- **Tkinter GUI** — four tabs (Transactions, Summary, Charts, Budgets & Goals), sortable table, live search, embedded matplotlib charts, HiDPI support.
- **JSON and CSV storage** — JSON for the complete state, CSV for spreadsheets; both validated record by record, with missing and corrupted file handling.
- **Budget alerts** — thresholds kept as constants (80 % / 100 %), sorted by severity and shown in every interface; goal milestones and missed deadlines too.
- **Streamlit dashboard** — sidebar filters (source, dates, categories, kind), KPIs with deltas, Plotly charts, heatmap, CSV upload and download.

8 Testing and quality

The project has **175 automated pytest tests**. Coverage is **98 % of the lines of the `finance_tracker` package**; the coverage scope *excludes* `gui_tkinter.py` (Tkinter) and `dashboard.py` (Streamlit), whose event code is exercised by smoke tests instead: the Tkinter window is built and clicked through on a real display, and the dashboard runs headlessly with Streamlit’s `AppTest`. The tests cover normal use, invalid input, missing and corrupted files, every pandas calculation on a hand-checked dataset, and complete CLI sessions driven by scripted answers. A standards test enforces Google-style docstrings, type hints and the concept tags, and `ruff` keeps the style consistent (`ruff check .` and `ruff format --check .` are clean).

9 Lessons learned and future work

Lessons. Validating data once, at the boundary (`__post_init__`), keeps the rest of the code simple. Choosing the right structure matters: a list keeps order, a dictionary gives fast lookup by name, a set removes duplicates. pandas replaces many loops with one readable `groupby`, but empty data must be

handled explicitly. Exception handlers are matched in order and by inheritance. Finally, looking at the real output found bugs that the unit tests missed — the “\$. . . \$” math-text problem and the HiDPI layout.

Future work. Recurring transactions generated automatically each month; import of bank statements with category suggestions; a SQLite backend for larger histories; spending forecasts in the dashboard; packaging as an installable application.